Lecture 6. Relational Data

Keys, mutating joins, filtering joins, set operations

Contents

6.1 Relational data **2**
6.1.1 Keys . . . 2

6.2 Mutating joins **2**
6.2.1 Duplicate keys . . . 3
6.2.2 Defining the key columns . . . 4

6.3 Filtering joins **5**

6.4 Set operations **5**

6.1 Relational data

Motivation. Real data rarely live in a single table. Flights refer to planes, airports and weather stored elsewhere, and questions such as "are delays worse in bad weather?" need columns from several tables at once. Joins combine tables through the variables they share.

Collections of multiple related tables are called **relational data**; the relations are always defined between *pairs* of tables. Besides `flights`, `nycflights13` contains

- `airlines`: full carrier names from their abbreviated codes;
- `airports`: information about each airport, identified by its `faa` code;
- `planes`: information about each plane, identified by `tailnum`;
- `weather`: the weather at each NYC airport for each hour.

`flights` connects to `planes` via `tailnum`, to `airlines` via `carrier`, to `airports` via `origin` and `dest`, and to `weather` via `origin` (the location) and `year`, `month`, `day`, `hour` (the time).

6.1.1 Keys

A **key** is a variable (or set of variables) that uniquely identifies an observation.

- A **primary key** identifies an observation in its own table, e.g. `planes$tailnum`.
- A **foreign key** identifies an observation in another table, e.g. `flights$tailnum`, which points to a row of `planes`.

It is worth verifying that a supposed primary key really is unique:

```
planes %>% count(tailnum) %>% filter(n > 1)
```

```
## # A tibble: 0 x 2
## # i 2 variables: tailnum <chr>, n <int>
```

```
weather %>% count(year, month, day, hour, origin) %>% filter(n > 1)
```

```
## # A tibble: 3 x 6
##    year month   day  hour origin     n
##   <int> <int> <int> <int> <chr>  <int>
## 1  2013    11     3     1 EWR        2
## 2  2013    11     3     1 JFK        2
## 3  2013    11     3     1 LGA        2
```

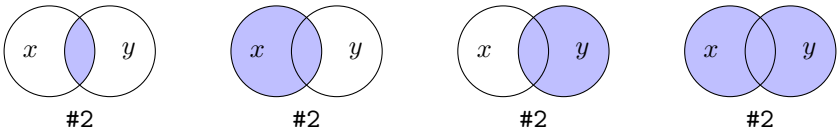
The second check finds a few duplicated hours (daylight-saving changes), so `weather` is not perfectly keyed [Supplement]. A primary key and the matching foreign key form a **relation**, typically one-to-many (each flight has one plane, each plane many flights).

6.2 Mutating joins

Motivation. A mutating join adds columns from one table to another, matching rows by key, much as mutate() adds computed columns.

We use two small tables to see exactly what each join does.

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  3, "x3")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2",  4, "y3")
```



Inner join: keep pairs of rows whose keys are equal; unmatched rows are dropped (dangerous, since observations disappear silently).

```
x %>% inner_join(y, by = "key")
```

```
## # A tibble: 2 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
```

Outer joins keep observations that appear in at least one table, filling missing columns with `NA`:

- `left_join(x, y)` keeps all rows of `x`;
- `right_join(x, y)` keeps all rows of `y`;
- `full_join(x, y)` keeps all rows of both.

```
left_join(x, y, by = "key")
```

```
## # A tibble: 3 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
```

```
full_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
## 4     4 <NA>  y3
```

The left join is the most common: use it to look up extra information without losing any original observation.

6.2.1 Duplicate keys

If only one table has duplicate keys (a one-to-many relation), each row of the other table is copied to every match:

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  2, "x3",  1, "x4")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2")
left_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     2 x3    y2
## 4     1 x4    y1
```

If both tables have duplicate keys, the result is every combination of matching rows (a Cartesian product within each key), which is usually an error; recent dplyr warns about an unexpected many-to-many relationship.

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  2, "x3",  3, "x4")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2",  2, "y3",  3, "y4")
left_join(x, y, by = "key")
```

```
## Warning in left_join(x, y, by = "key"): Detected an unexpected many-to-many relationship
    between `x` and `y`.
## i Row 2 of `x` matches multiple rows in `y`.
## i Row 2 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to
##   silence this warning.
```

```
## # A tibble: 6 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     2 x2    y3
## 4     2 x3    y2
## 5     2 x3    y3
## # i 1 more row
```

6.2.2 Defining the key columns

On the real data, start from a narrower table so the added columns are visible:

```
flights2 <- flights %>% select(year:day, hour, origin, dest, tailnum, carrier)
```

- `by = NULL` (default, a **natural join**) uses all variables that appear in both tables, and reports which:

```
flights2 %>% left_join(weather)
```

```
## # A tibble: 336,776 x 18
##    year month   day  hour origin dest  tailnum carrier  temp  dewp humid wind_dir
##   <int> <int> <int> <dbl> <chr>  <chr> <chr>   <chr>   <dbl> <dbl> <dbl>    <dbl>
## 1  2013     1     1     5 EWR    IAH   N14228  UA       39.0  28.0  64.4      260
## 2  2013     1     1     5 LGA    IAH   N24211  UA       39.9  25.0  54.8      250
## 3  2013     1     1     5 JFK    MIA   N619AA  AA       39.0  27.0  61.6      260
## 4  2013     1     1     5 JFK    BQN   N804JB  B6       39.0  27.0  61.6      260
## 5  2013     1     1     6 LGA    ATL   N668DN  DL       39.9  25.0  54.8      260
## # i 336,771 more rows
## # i 6 more variables: wind_speed <dbl>, wind_gust <dbl>, precip <dbl>, pressure <dbl>,
## #   visib <dbl>, time_hour <dttm>
```

- `by = "tailnum"` uses only that common variable. `flights` and `planes` both have a `year` column with different meanings (flight year, manufacturing year), so a natural join would be wrong; with `by = "tailnum"` the two years are kept and disambiguated as `year.x`, `year.y`.
- `by = c("dest" = "faa")` matches a variable with different names in the two tables: `dest` in `flights2` and `faa` in `airports`. Joining on `c("origin" = "faa")` instead would add the origin airport's information.

```
flights2 %>% left_join(airports, c("dest" = "faa")) %>% select(dest, name, lat, lon)
```

```
## # A tibble: 336,776 x 4
##   dest  name                            lat   lon
##   <chr> <chr>                         <dbl> <dbl>
```

```
## 1 IAH   George Bush Intercontinental     30.0 -95.3
## 2 IAH   George Bush Intercontinental     30.0 -95.3
## 3 MIA   Miami Intl                       25.8 -80.3
## 4 BQN   <NA>                             NA    NA
## 5 ATL   Hartsfield Jackson Atlanta Intl  33.6 -84.4
## # i 336,771 more rows
```

6.3 Filtering joins

Motivation. Sometimes we do not want any new columns, only to keep or drop rows according to whether they have a match in another table.

Filtering joins affect rows, not columns. To restrict flights to the ten most popular destinations:

```
top_dest <- flights %>% count(dest, sort = TRUE) %>% head(10)
```

- `semi_join(x, y)` keeps the rows of `x` that *have* a match in `y`. It works for any number of key variables, where `filter(dest %in% top_dest$dest)` works only for one.
- `anti_join(x, y)` keeps the rows of `x` that *do not* have a match. It is the main tool for diagnosing join mismatches.

```
semi_join(flights, top_dest, by = "dest") %>% nrow()
```

```
## [1] 141145
```

```
flights %>%
  anti_join(planes, by = "tailnum") %>%
  count(tailnum, sort = TRUE)
```

```
## # A tibble: 722 x 2
##   tailnum     n
##   <chr>   <int>
## 1 <NA>     2512
## 2 N725MQ    575
## 3 N722MQ    513
## 4 N723MQ    507
## 5 N713MQ    483
## # i 717 more rows
```

Many flights have no plane metadata; the missing tail numbers mostly belong to two carriers (AA and MQ), which report fleet numbers rather than tail numbers [Supplement].

Checking a join [Supplement]

Before joining: identify the primary key from an understanding of the data (not by searching for a unique combination); check that no key value is missing; and check with `anti_join()` that every foreign key matches. Compare the number of rows before and after a left join: if it grows, the key was not unique.

6.4 Set operations

These treat every row as a whole, comparing all variables:

```
df1 <- tribble(~x, ~y,  1, 1,  2, 1)
df2 <- tribble(~x, ~y,  1, 1,  1, 2)
intersect(df1, df2)   # rows in both
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     1
```

```
union(df1, df2)        # unique rows in either
```

```
## # A tibble: 3 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     1
## 2     2     1
## 3     1     2
```

```
setdiff(df1, df2)      # rows in df1 but not in df2
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     2     1
```

```
setdiff(df2, df1)
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     2
```